

On the Role of Proposal Support in Diffusion-Based Offline Reinforcement Learning for ARC-AGI Sequential Decision-Making

Yunho Kim^{1*}, Jaehyun Park¹, Heejun Kim¹, Sejin Kim¹,
Byung-Jun Lee², Sundong Kim¹

¹*Department of AI Convergence, Gwangju Institute of Science and
Technology, Gwangju, Republic of Korea.

²Department of Artificial Intelligence, Korea University, Seoul,
Republic of Korea.

*Corresponding author(s). E-mail(s): rladbsgh797210@gmail.com;
Contributing authors: jaehyun00518@gmail.com; ya2298yay@gmail.com;
sjkim7822@gmail.com; byungjunlee@korea.ac.kr; sundong@gist.ac.kr;

Abstract

Proposal-selection methods in offline reinforcement learning (RL) make decisions by generating candidate behaviors and selecting among them with learned value estimates. In multi-step settings, however, failures are difficult to attribute to proposal generation, value-based selection, or their interaction. We study this issue in ARC-AGI, formulated as a sequential decision-making environment with intermediate transitions. We introduce Synthesized Offline Learning data for Abstraction and Reasoning (SOLAR), which converts ARC-AGI tasks into validated state-action trajectories with controlled variation in trajectory quality. Using Latent Diffusion-Constrained Q-Learning (LDCQ), we find that performance is strong when solution-relevant behaviors are covered by the proposal distribution but degrades sharply when they are absent. To separate proposal and selection errors, we use ground-truth latents as oracle references and test whether the Q-function selects them when present. This analysis shows that value-based selection remains comparatively reliable even as proposal coverage drops, identifying limited proposal support as the main bottleneck in diffusion-based offline RL for ARC-AGI.

Keywords: offline RL, diffusion models, sequential decision-making, ARC-AGI, reasoning

1 Introduction

Offline Reinforcement Learning (RL) aims to learn decision-making policies from fixed datasets without additional environment interaction. Some recent offline RL methods follow a proposal–selection structure, in which a generative model proposes candidate behaviors and a learned value function selects among them. This structure is most explicit in diffusion-based approaches such as LDCQ (Venkatraman et al. 2024), where a diffusion model generates diverse multi-step behavior candidates and a value function selects the candidate with the highest estimated return before execution.

However, standard offline RL evaluations are not well suited for diagnosing failures within this proposal–selection decomposition. Common benchmark suites, including D4RL (Fu et al. 2020) tasks based on MuJoCo locomotion and robotic manipulation environments, primarily evaluate continuous-control performance through cumulative return rather than step-level correctness. Thus, when an agent fails, these benchmarks provide limited basis for determining whether the error comes from the proposal model, the value-based selector, or their interaction over multiple steps. Because intermediate states lie on a dense manifold, deviation from the optimal path often still yields high return, making step-level failure attribution intractable.

This diagnostic gap becomes especially important in structured reasoning tasks, where solutions must be constructed through discrete intermediate transformations and each step can be evaluated against a verifiable solution process. ARC-AGI (Chollet 2019) provides a natural setting for studying this issue. Given executable solving trajectories, each operation can be evaluated against the corresponding ground-truth transition, making step-level attribution tractable. This discrete, verifiable structure makes it possible to directly ask whether a failure at a given step was caused by the absence of the correct behavior in the proposal set, or by the value function’s failure to select it when present.

Despite its suitability, the original ARC-AGI benchmark specifies only input–output pairs and does not provide executable solving trajectories, which prevents direct analysis of intermediate decision processes. To bridge this gap, we introduce Synthesized Offline Learning data for Abstraction and Reasoning (SOLAR)¹, a trajectory-based dataset that converts ARC-AGI tasks into validated offline RL datasets with executable state–action trajectories. SOLAR provides two properties that are essential for diagnostic analysis: (i) ground-truth state–action trajectories that define what constitutes a correct intermediate behavior, and (ii) controlled trajectory quality that allows systematic variation of the training distribution. Together, these properties make it possible to separate proposal and selection errors in a way that continuous-control benchmarks cannot support.

Using Latent Diffusion Constrained Q-learning (LDCQ) (Venkatraman et al. 2024), we show that performance degrades when correct behaviors are absent from the proposal set, and that oracle candidate analysis attributes most of this degradation to proposal coverage rather than to value-based selection. In compositional settings, the same limitation manifests as a failure to bridge unseen intermediate states, even when component transformations have been learned individually.

¹Codes are publicly available at https://github.com/GIST-DSLab/solar_ldcq.

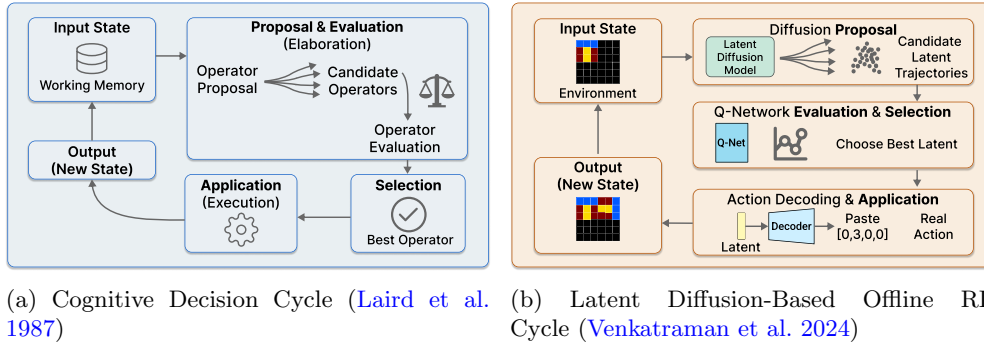


Fig. 1: A staged decision motif studied in this paper. Both cycles can be decomposed into the phases of proposal, evaluation, selection, and application (execution). Mapping LDCQ onto the cognitive architecture framing makes each component’s role explicit and enables separate diagnosis of proposal and selection failures.

Our contributions are summarized as follows:

- We identify the verifiable discrete structure of ARC-AGI as a key property that enables step-level diagnosis of proposal–selection failures, which is not tractable in continuous-control benchmarks.
- We introduce SOLAR, a trajectory-based dataset providing ground-truth executable trajectories, controlled trajectory-quality variation, and oracle candidate analysis for ARC-AGI tasks.
- We use SOLAR to diagnose LDCQ at the step level via controlled trajectory distributions and oracle candidate analysis, separating proposal coverage from value-based selection.
- We show that proposal support is a key bottleneck for robustness and compositional generalization in diffusion-based offline RL, while value-based selection remains comparatively reliable when correct candidates are present.

2 Background

2.1 Proposal–Selection in Offline RL

We analyze LDCQ (Venkatraman et al. 2024) through a proposal–selection lens. As illustrated in Figure 1, LDCQ’s decision cycle (encoding candidate behaviors, evaluating them with a Q-function, selecting the best, and executing) maps naturally onto the staged cycle of proposal, evaluation, selection, and execution originally described in cognitive architectures (Laird et al. 1987). We adopt this framing to make the role of each component explicit and to enable separate diagnosis of proposal and selection failures. Trajectory segments $\tau_t = (s_t, a_t, \dots, s_{t+H-1}, a_{t+H-1})$ are encoded into latent variables z_t , which represent H -step behaviors.

Latent Representation Learning.

A β -VAE is trained to encode trajectory segments into latent variables $z_t \sim q_\phi(z_t | \tau_t)$ and decode actions via a policy $\pi_\theta(a | s, z_t)$. The model is trained to learn compact representations of multi-step behaviors.

Latent Proposal via Diffusion.

A conditional diffusion model $p_\psi(z_t | s_t)$ is trained over latent variables to generate candidate behaviors given the current state. Under our framing, this serves as the proposal distribution in the decision pipeline, and its coverage over solution-relevant behaviors is central to the analysis in this paper.

Value-based Selection.

A Q-function $Q(s_t, z_t)$ is trained to evaluate latent behaviors over an H -step horizon. Given a transition from s_t to s_{t+H} , the target value is computed by sampling candidate next latents from the diffusion prior and selecting the one with the highest Q-value:

At inference time, multiple candidates are sampled from $p_\psi(z | s_t)$ and the highest-valued one is selected, enabling sequential decision-making through iterative proposal and selection.

2.2 ARC-AGI Data Augmentation Methods

ARC-AGI (Chollet 2019) is a benchmark where agents must infer transformation rules from a small number of demonstration input–output pairs (typically 3) and apply the inferred rule to a held-out test input. RE-ARC (Hodel 2024) synthesizes diverse task instances using a domain-specific language of 141 primitives, and BARC (Li et al. 2024) uses large language models to generate Python programs that solve ARC tasks, constructing a large-scale dataset of input–output pairs. Both approaches support generalization evaluation but provide no intermediate execution states. SOLAR differs in purpose: rather than generating task variants, it synthesizes full state–action trajectories using the ARC-LE action space, enabling offline RL training and the step-level failure analysis that is the focus of this paper.

2.3 ARC-AGI as a Structured Decision Environment

We formulate ARC-AGI as a Markov Decision Process using ARC-LE (Lee et al. 2024), a reinforcement learning environment that provides explicit state transitions and a discrete action space. The state s_t comprises the current working grid, the input grid, and the clipboard. The demonstration input–output examples are fixed throughout the episode and provided as task conditioning to all model components. The action a_t consists of an operation opr_t and a bounding-box selection $sel_t \in \mathbb{Z}^4$ with coordinates (x, y, h, w) (see Appendix B.2), with the transition:

$$s_{t+1} = f(s_t, a_t), \quad a_t = (opr_t, sel_t). \quad (1)$$

A reward of 1 is provided only upon correct submission. All other steps yield 0.

3 SOLAR: Trajectory Data for Step-Level Analysis

We introduce the Synthesized Offline Learning data for Abstraction and Reasoning (SOLAR), a trajectory-based dataset that converts ARC-AGI tasks into validated offline RL datasets. Most existing approaches to ARC-AGI evaluate only final answer correctness, without capturing where execution fails within a sequential decision pipeline. SOLAR addresses this by providing explicit state-action trajectories that enable stage-wise failure diagnosis, with two properties for step-level diagnostic analysis: (i) ground-truth trajectories that define correct intermediate behavior at each step, and (ii) controlled trajectory quality that enables systematic variation of the training distribution.

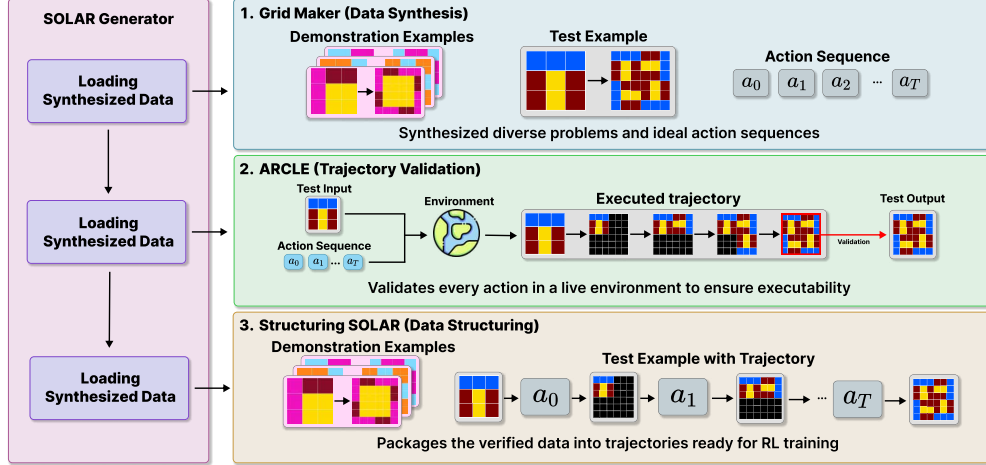


Fig. 2: Dataset construction process with SOLAR-Generator. (i) Input-output pairs and action sequences are synthesized, (ii) validated through execution in ARCLE, and (iii) converted into json-style episodes for offline RL.

3.1 Dataset Construction

As illustrated in Figure 2, SOLAR is constructed through three stages: (i) synthesizing input-output pairs and action sequences using Grid Maker, (ii) validating every action in a live ARCLE (Lee et al. 2024) environment to ensure executability, and (iii) packaging the verified data into trajectory episodes for offline RL training. Each SOLAR task consists of demonstration input-output pairs and a test example with an executable episode. An episode is represented as $\tau = \{(s_t, a_t, s_{t+1}, r_t)\}_{t=0}^T$, where r_t is a sparse reward provided only upon correct submission via the **Submit** action. Execution-level trajectory information is provided only for the test instance, maintaining fidelity to the original ARC-AGI setting while enabling step-by-step observation of the intermediate solving process.

3.2 Trajectory Generation

SOLAR is constructed using a rule-based synthesis procedure implemented in SOLAR-Generator. For each task, a solving action sequence is first synthesized to correctly transform the input into the target output, then validated in ARCLE. To increase diversity, additional trajectories are generated by varying action orderings or alternative combinations of operations, resulting in multiple valid trajectories per task ranging from direct to longer paths.

SOLAR supports controlled synthesis of expert and suboptimal trajectories. Suboptimal trajectories share a prefix with an expert trajectory and deviate at a random step via two task-plausible perturbations: (i) modifying the selection while keeping the operation fixed, or (ii) replacing a color-changing operation with an alternative while preserving the selection. These perturbations are motivated by error patterns observed in human ARC solving (Kim et al. 2026), keeping suboptimal trajectories close to expert behavior and reflecting realistic mistakes rather than arbitrary noise. Across datasets, the task distribution, environment dynamics, and data budget are held fixed. Only the proportion of expert and suboptimal trajectories varies, attributing performance differences directly to dataset composition. Additional details are provided in Appendix B.

4 Experiments

4.1 Experimental Setup

All experiments are conducted using the SOLAR dataset, which provides executable state-action trajectories for ARC-AGI tasks.

To control the trajectory distribution, we construct offline datasets by mixing expert trajectories $\mathcal{D}_{\text{exp}}$ and suboptimal trajectories $\mathcal{D}_{\text{sub}}$:

$$\mathcal{D}(\alpha) = \alpha\mathcal{D}_{\text{exp}} + (1 - \alpha)\mathcal{D}_{\text{sub}}, \quad (2)$$

where $\alpha \in [0, 1]$ controls the proportion of expert trajectories. Suboptimal trajectories may either reach the correct solution via a longer path or fail to reach it entirely. We consider $\alpha = 1$ (expert-only) and $\alpha = 1/2$ (mixed-quality) settings.

For each task, we synthesize 5,000 training episodes and 100 test episodes using SOLAR-Generator, with non-overlapping test examples. We report task success rate, defined as the proportion of episodes that terminate with a correct final output.

4.2 Behavior under Expert Trajectories

Under expert-only trajectories, LDCQ solves most tasks with high accuracy, though performance varies by task structure (Table 1). Several tasks exceed 90% accuracy, while 0d3d703e achieves 0% despite expert-only training. One possible explanation is that this task requires a global color permutation rule that is not recoverable from individual intermediate transitions, making step-by-step trajectory-level solving insufficient even when training data is clean. In contrast, tasks involving direct spatial transformations appear more tractable, as each step produces a locally observable

state change. For such tasks, the diffusion prior can learn a concentrated proposal distribution over correct behaviors, since the mapping from state to correct action is locally consistent throughout the episode.

Table 1: Single-task performance with expert-only trajectories.

Task ID	Accuracy	Task ID	Accuracy
5c0a986e-colorfix	98.7 $\pm$ 2.31%	a65b410d	94.7 $\pm$ 5.51%
4c4377d9	91.7 $\pm$ 9.29%	6f8cd79b	91.0 $\pm$ 0.00%
4258a5f9	72.7 $\pm$ 10.97%	007bbfb7	77.7 $\pm$ 1.15%
46442a0e	74.3 $\pm$ 2.89%	5c0a986e-colordiff	64.0 $\pm$ 8.72%
74dd1130	27.0 $\pm$ 4.36%	0d3d703e	0.0 $\pm$ 0.00%

4.3 Selecting Efficient Solution Paths

We examine whether LDCQ can identify more efficient behaviors when multiple solutions exist. To study this, we construct a controlled SOLAR dataset for task a65b410d,

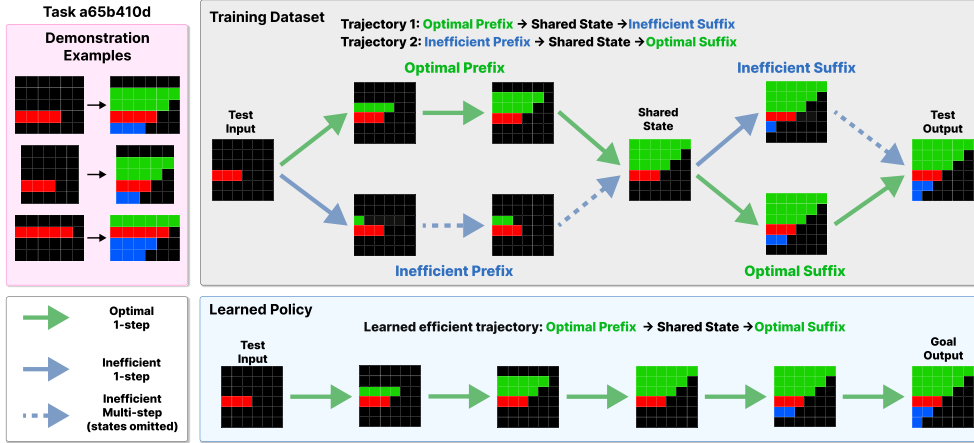


Fig. 3: Two training trajectories for Task a65b410d. The trajectories share an intermediate state, with each being optimal in one half and inefficient in the other. LDCQ policy finds an efficient trajectory by combining the optimal segments from both.

where all trajectories reach the correct solution but differ in efficiency. As illustrated in Figure 3, the two training trajectories diverge before a shared intermediate state and reconverge after it. One takes an optimal path in the first half but an inefficient path in the second, and the other does the reverse. As a result, no single training trajectory is fully optimal end-to-end, and a fully efficient solution can only be constructed by combining efficient segments from different trajectories.

We compare LDCQ against two ablated baselines that omit selection: VAE Prior and Diffusion Prior, which use a single candidate sampled from the VAE and diffusion model respectively. As shown in Table 2, all methods achieve high accuracy, but LDCQ achieves this with fewer steps.

Table 2: Comparison of policies on a task where all trajectories reach correct solutions but differ in efficiency. LDCQ achieves the same accuracy in fewer than half the steps.

Policy	Accuracy	Avg. Steps
VAE Prior	90%	19.4
Diffusion Prior	99%	15.4
LDCQ (w/ Q)	100%	7.0

While generative policies (VAE prior, Diffusion prior) tend to follow training trajectories and produce longer solutions, LDCQ achieves the same accuracy in fewer than half the steps (7.0 vs. 15.4–19.4). This suggests that value-based selection enables the model to prefer more efficient action sequences rather than simply reproducing observed trajectories.

4.4 Behavior under Mixed-quality Trajectories

We compare LDCQ against BC as a non-value-based reference, and against CQL (Kumar et al. 2020) and IQL (Kostrikov et al. 2022) as representative offline RL methods that apply explicit action-level constraints rather than latent-level proposals. This allows us to assess whether LDCQ’s latent proposal approach offers a distinct advantage under clean data and whether it introduces additional sensitivity under mixed-quality conditions. Mixed-quality training degrades performance, and LDCQ shows a larger drop than CQL or IQL despite leading under expert-only data.

Table 3: Baseline comparison on multi-task learning under expert-only ($\alpha = 1$) and mixed-quality ($\alpha = 1/2$) training.

Method	Expert-only	Mixed-quality
BC	38.8±1.86%	9.67±2.44%
CQL (Kumar et al. 2020)	61.9±0.31%	37.5±0.31%
IQL (Kostrikov et al. 2022)	58.3±0.31%	32.4±1.11%
LDCQ (Venkatraman et al. 2024)	66.9±6.39%	23.7±6.04%

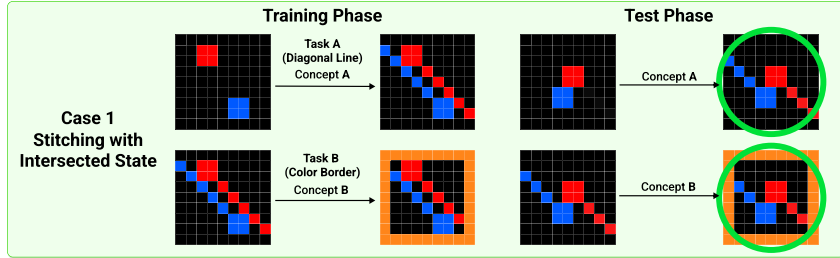
As shown in Table 3, LDCQ achieves the highest performance under expert-only data, but its performance drop under mixed-quality trajectories is notably larger than that of other methods. This higher sensitivity to distributional variation may be attributed to the nature of latent-based proposal generation. Unlike CQL and IQL, which apply explicit action-level constraints or regularization, LDCQ relies

on a diffusion prior over latent trajectory segments and is therefore more susceptible to degradation when the training distribution contains suboptimal behaviors. Action-level methods can remain conservative by penalizing out-of-distribution actions regardless of trajectory quality, whereas LDCQ’s proposal quality is tied directly to the training distribution: suboptimal trajectories shift the diffusion prior toward suboptimal latent regions, reducing the probability that correct behaviors appear in the candidate set. Qualitatively, the agent often reaches states close to the target but fails to complete the task, producing inconsistent action sequences.

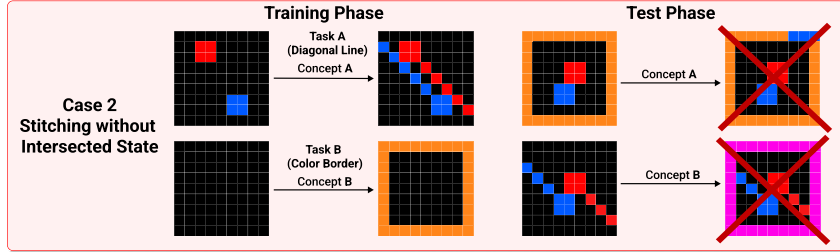
These results suggest that decision quality is closely tied to the availability of suitable candidate behaviors in the proposal stage. In Section 5, we isolate this effect directly. Using ground-truth latents as oracle references, we test whether the Q-network selects correctly when the oracle candidate appears among proposals, separating proposal coverage from selection accuracy.

4.5 Compositional Generalization

Compositional generalization requires shared intermediate states at the composition boundary. Without such coverage, the agent fails even when each component transformation has been learned individually. We train on Task A (diagonal line) and Task



(a) Applying learned concepts to seen intermediate states (Case 1)



(b) Applying learned concepts to unseen intermediate states (Case 2)

Fig. 4: Compositional generalization with and without intermediate state support. (a) When the intermediate state at the composition boundary is covered by training data, the agent composes the two learned transformations. (b) When absent, composition fails despite each transformation being learned individually, highlighting that generalization is constrained by trajectory-level support rather than concept-level learning.

Table 4: Effect of trajectory support on compositional generalization.

Condition	State Seen	Transition Seen	Accuracy
Seen task	Yes	Yes	100%
Unseen state	No	Partial	0–33%
Composition (with shared states)	Yes	Yes	100%
Composition (without shared states)	No	No	0%

B (color border) separately, then evaluate their sequential composition. Two conditions are tested: Case 1, where the intermediate state after Task A appears in Task B’s training data, and Case 2, where it does not. As a baseline, each transformation is evaluated individually on unseen intermediate states (Case 2 without composition); performance already degrades before composition is attempted, as detailed in Table 4.

As shown in Figure 4 and Table 4, composition succeeds in Case 1 where the intermediate state is covered by training data, but fails in Case 2 where it is not, even though each transformation has been learned individually. When transitions are only partially supported, performance varies across concepts, with Concept A failing completely (0%) and Concept B achieving limited success (33%).

These results indicate that compositional generalization is structurally constrained by the availability of compatible trajectory segments in the proposal distribution: even when individual transformations are learned, the agent cannot compose them if the necessary intermediate states and transitions are absent from the training data.

5 Analysis

Limited proposal support is the primary constraint on LDCQ’s performance. The proposal set often lacks correct candidates, and this scarcity makes value-based selection harder rather than constituting a separate failure. We establish this by decomposing failures across the proposal and selection stages and by measuring selection accuracy conditional on the correct candidate being present among the proposals.

5.1 Failure Decomposition

Under mixed-quality data, LDCQ exhibits marked performance degradation. Early-stage transitions are often reasonable, but errors accumulate over steps, and generated episodes frequently contain locally valid transitions that fail to form coherent sequences toward the goal. Failures may originate in the proposal stage, the selection stage, or both. The two stages are coupled during inference. A correct behavior that is absent from the proposal set cannot be selected regardless of Q-function quality. As a result, standard evaluation cannot directly attribute failure to a single stage. Disentangling the two requires holding one constant while varying the other, which the following oracle analysis provides.

5.2 Diagnosing the Source of Failure

To identify the source of failure, we analyze the proposal and selection stages separately. LDCQ follows a proposal–selection pipeline, where candidate behaviors are generated from a diffusion prior and evaluated using a learned Q-function.

We first examine whether the Q-network fails to correctly select among candidate behaviors. To isolate this effect, we perform an oracle candidate analysis using the ground-truth latent as a reference to assess both proposal coverage and selection reliability. Since SOLAR provides ground-truth trajectories for test instances, we encode each trajectory segment with the trained β -VAE to obtain the oracle latent representation. For each of 500 randomly sampled test states, we generate 100 latent candidates from the diffusion prior and measure two quantities. The first is the rate at which the oracle latent appears among the proposals. The second is the Q-network’s selection accuracy when the oracle is present.

Table 5: Proposal coverage and conditional selection accuracy.

	Expert-only	Mixed-quality
$P(\text{oracle} \in \text{proposals})$	95.3%	77.7%
$P(\text{correct action} \mid \text{oracle})$	91.2%	81.8%

As shown in Table 5, the probability that the correct candidate is included in the proposal set decreases notably under mixed-quality data (95.3% $\rightarrow$ 77.7%). When the correct candidate is available, however, the Q-network still selects it with reasonably high accuracy. Conditional selection accuracy thus falls much less than coverage (91.2% $\rightarrow$ 81.8%), so coverage degrades more than selection. Crucially, the two stages are not independent. The Q-function is trained over the diffusion-proposed latent distribution, so when proposal quality degrades, selection becomes harder as well. The residual drop in conditional accuracy is therefore better read as a downstream effect of weakened proposal support than as a separate failure of value estimation. In other words, the limited support of the proposal set is the dominant bottleneck, and it also makes selection appear less reliable than it is under clean data.

5.3 Proposal Support Limitation

Having traced the failure to weak proposal support, we next quantify how thin this support actually is, and whether it can be compensated by sampling more proposals.

For each of the 500 test states, we count how many of the 100 diffusion proposals match the oracle latent. This yields a per-state oracle count, where 0 means no correct candidate and 100 means all proposals are correct. Table 6 reports the distribution. Under mixed-quality data, the correct behavior is not merely present or absent but typically scarce. In fact, 22.3% of states have no oracle proposal at all, and a further 34.1% have between 1 and 20. More than half of all states (56.4%) thus carry at most 20 correct candidates out of 100. The nominal coverage of 77.7% therefore overstates the effective support. Even when a correct candidate is technically present, it is so rare

among the 100 proposals that the Q-network seldom ranks it highest, and selection is forced among suboptimal options.

Table 6: Distribution of oracle candidates across states (100 proposals); many states lack any correct candidate under mixed-quality data. Endpoints 0 and 100 denote no-coverage and full-coverage; intermediate ranges are grouped by density.

# Oracle proposals	Expert-only	Mixed-quality
0	4.7±0.63%	22.3±1.54%
1–20	2.1±0.67%	34.1±1.16%
21–50	2.5±0.34%	13.0±1.35%
51–99	14.6±1.92%	16.3±1.63%
100	76.0±2.23%	14.3±1.39%

We next ask whether sampling more proposals compensates for low coverage. Running standard LDCQ inference on the multi-task mixed-quality setting ($\alpha = 1/2$), we vary the number of diffusion proposals from 1 to 1000 and select the best-valued candidate by the Q-function.

Table 7: Effect of the number of proposals on performance ($\alpha = 1/2$).

# Proposals	Accuracy (%)	# Proposals	Accuracy (%)
1	5.6	100	25.4
10	18.2	500	20.0
50	26.2	1000	19.8

As shown in Table 7, accuracy peaks at 50 proposals (26.2%) and degrades beyond. The gain up to 50 reflects better coverage. Beyond it, the diffusion prior’s limited support means additional samples come from peripheral latent regions rather than correct solutions. Such out-of-distribution candidates can receive inflated Q-values, consistent with overestimation bias in offline RL, causing selection to favor unreliable behaviors over correct but lower-scored ones. Thus sampling more candidates does not simply widen coverage. It also exposes the selection stage to more out-of-distribution proposals, so the two stages fail together rather than independently.

This indicates that proposal limitations arise not only from insufficient coverage, but also from the quality of the proposal distribution. Together, these results reveal a structural limitation of the LDCQ framework. Decision quality is fundamentally constrained by the support and quality of the proposal distribution. Even a well-trained value function cannot recover correct behavior when suitable candidates are absent or poorly represented. This trade-off suggests that improving proposal quality, rather than simply increasing proposal count, is the key challenge for diffusion-based offline RL under mixed-quality data.

6 Discussion

Our analysis reveals that decision quality in diffusion-based offline RL is fundamentally constrained by the support of the proposal distribution, manifesting in three related failure modes: insufficient coverage under mixed-quality data, degraded selection under out-of-distribution proposals, and failure of compositional generalization at unseen intermediate states.

Proposal coverage and selection reliability as coupled constraints.

Training $p_\psi(z_t \mid s_t)$ to replicate the offline dataset means the proposal distribution inherits its limitations directly. When data is incomplete or noisy, correct behaviors are absent from the candidate set and cannot be recovered regardless of value function quality. Sampling more candidates does not resolve this.

Limitations: compositional generalization and offline learning.

Compositional generalization fails entirely when intermediate states at the composition boundary are absent from training data, indicating that the trajectory-level latent representation is a structural barrier to flexible recombination. At a broader level, the proposal distribution is bounded by the support of the offline dataset, and correct behaviors cannot be recovered when absent from training data regardless of value function quality. This is an open problem that purely offline approaches cannot resolve without richer data or limited online interaction.

Scope of evaluation.

Our experiments use limited ARC-AGI tasks. Training across a larger task set simultaneously is constrained by generalization difficulty and training time, as the agent must infer diverse transformation rules from limited demonstrations. The primary contribution of this paper is a methodology for measuring proposal quality and diagnosing failure modes, rather than a claim of generalization across the full ARC-AGI benchmark.

7 Conclusion

We studied the proposal–selection decomposition in diffusion-based offline RL through the lens of ARC-AGI, a structured reasoning domain where each intermediate action can be evaluated against a ground-truth trajectory. To enable this analysis, we introduced SOLAR, a trajectory-based dataset providing controlled variation in data quality and oracle candidate analysis. Our experiments demonstrate that performance degradation in LDCQ arises primarily from insufficient proposal support, since a well-trained Q-function cannot recover a correct behavior that is absent from the candidate set. These findings identify proposal quality as the principal bottleneck for diffusion-based offline RL in structured sequential decision-making, and SOLAR provides a reusable methodology for diagnosing such failures.

A Training Details

Training LDCQ requires three sequential stages, each building on the outputs of the previous. First, a β -VAE is trained to encode multi-step trajectory segments into compact latent representations and decode actions from them. Second, a conditional latent diffusion model is trained to generate candidate latents given the current state, serving as the proposal distribution at inference time. Third, a Q-network is trained using latents sampled from the diffusion model to evaluate and select among candidate behaviors. Figure 5 illustrates each stage and the inference procedure.

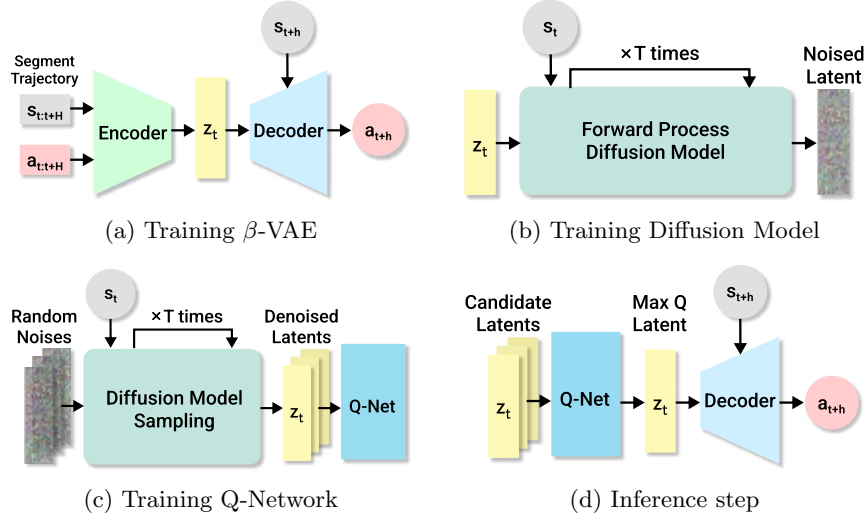


Fig. 5: (a)–(c) Training stages of LDCQ. (a) Training a β -VAE with an encoder that encodes H -horizon segment trajectories into latents z_t , and a policy decoder that decodes actions based on z_t and state s_{t+h} where $h \in [0, H)$. (b) Training a diffusion model based on z_t and s_t . (c) Training a Q-network using latents sampled through the diffusion model. (d) LDCQ inference step: possible latents at s_t are sampled through the diffusion model, and the agent executes actions from the highest Q-value latent.

Training Latent Encoder and Policy Decoder

As shown in Figure 5a, the encoder q_ϕ is trained to encode H -length trajectory segments τ_t into latent representations z_t , and the policy decoder π_θ decodes actions a_{t+h} given state s_{t+h} and latent z_t , where $h \in [0, H)$. The β -VAE is trained by minimizing the ELBO loss, which consists of the action reconstruction loss and the KL divergence between the approximate posterior $q_\phi(z_t|\tau_t)$ and the prior $p_\omega(z_t|s_t)$:

$$\mathcal{L}(\theta, \phi, \omega) = -\mathbb{E}_{\tau_t \sim \mathcal{D}} \left[\mathbb{E}_{q_\phi(z_t|\tau_t)} \left[\sum_{l=t}^{t+H-1} \log \pi_\theta(a_l|s_l, z_t) \right] - \beta \cdot D_{KL}(q_\phi(z_t|\tau_t) \| p_\omega(z_t|s_t)) \right] \quad (3)$$

Training Latent Diffusion Model

The diffusion model $p_\psi(\mathbf{z}_t|\mathbf{s}_t)$ is trained on $(\mathbf{s}_t, \mathbf{z}_t)$ pairs to learn the denoising function $\mu_\psi(\mathbf{z}_t^j, \mathbf{s}_t, j)$, where $j \in [0, T]$ is the diffusion timestep. Following (Venkatraman et al. 2024), we predict the original latent rather than the noise, using the Min-SNR- γ strategy (Hang et al. 2023) to balance the loss across timesteps:

$$\mathcal{L}(\psi) = \mathbb{E}_{j \sim [1, T], \tau_H \sim \mathcal{D}, \substack{\mathbf{z}_t \sim q_\phi(\mathbf{z}_t | \tau_t), \\ \mathbf{z}_t^j \sim q(\mathbf{z}_t^j | \mathbf{z}_t^0)}} \left[\min\{\text{SNR}(j), \gamma\} \cdot \|\mathbf{z}_t^0 - \mu_\psi(\mathbf{z}_t^j, \mathbf{s}_t, j)\|^2 \right] \quad (4)$$

Training Q-Network

We adopt DDIM sampling (Song et al. 2020) instead of DDPM (Ho et al. 2020) to accelerate latent generation. The Q-network is trained on $(\mathbf{s}_t, \mathbf{z}_t, r_{t:t+H}, \mathbf{s}_{t+H})$ tuples using Clipped Double Q-learning (Fujimoto et al. 2018) with a Prioritized Experience Replay buffer (Schaul et al. 2016). Since ARC-AGI tasks require inferring analogies from demonstration pairs, the embedded demonstration representation $\mathbf{p}_{emb}$ is additionally conditioned on in the Q-function:

$$Q(\mathbf{s}_t, \mathbf{z}_t, \mathbf{p}_{emb}) \leftarrow \left(r_{t:t+H} + \gamma^H Q\left(\mathbf{s}_{t+H}, \underset{\mathbf{z} \sim p_\psi(\mathbf{z}_{t+H} | \mathbf{s}_{t+H})}{\text{argmax}} Q(\mathbf{s}_{t+H}, \mathbf{z}, \mathbf{p}_{emb}), \mathbf{p}_{emb} \right) \right) \quad (5)$$

A.1 Hyperparameters

We used a horizon length of 5 for encoding skill latents, meaning the model plans and evaluates actions over a five-step lookahead.

We trained the diffusion model with 500 diffusion steps. If the number of diffusion steps is too small, it can lead to high variance in the sampling process, potentially causing errors during the decoding of operations or selections in ARCLE. To minimize these errors, we set the number of diffusion steps to 500, ensuring more accurate operation and selection decoding from the sampled latents.

We set the discount factor to 0.5 to ensure the model appropriately balances immediate and future rewards. Since the total steps required to reach the correct answer in ARCLE are usually fewer than 20, a high discount factor could cause the agent to struggle in distinguishing between submitting at the correct state and continuing with additional steps, which could lead to episode failure.

The hyperparameters that we used for training three stages of LDCQ are shown in Tables 8, 9 and 10.

A.2 Hardware

We used an NVIDIA A100-SXM4-40GB GPU to train the model. Training the β -VAE took about 7 hours, while training the diffusion model and Q-network each took around 6 to 10 hours.

Table 8: Hyperparameters for β -VAE training

Parameter	Value
Learning rate	5e-5
Batch size	128
Epochs	400
Horizon (H)	5
Latent dimension (z)	256
KL loss ratio (β)	0.1
Hidden layer dimension	512

Table 9: Hyperparameters for diffusion model training

Parameter	Value
Learning rate	1e-4
Batch size	64
Epochs	400
Diffusion steps (T)	500
Variance schedule	linear
Sampling algorithm	DDIM
γ (Min-SNR- γ)	5

Table 10: Hyperparameters for training DQN (Q-Network)

Parameter	Value
Learning rate	5e-4
Batch size	64
Discount factor (γ)	0.5
Target net update rate (ρ)	0.995
PER buffer α	0.7
PER buffer β range	[0.3, 1)
PER buffer β increment	+0.03 per 2,000 steps
Diffusion samples for batch argmax	100

B Details of SOLAR-Generator

B.1 Design Rationale for SOLAR

We develop SOLAR as the trajectory-based dataset for this study for three reasons.

Controlled mixed-quality trajectory generation based on realistic human error patterns.

SOLAR-Generator synthesizes expert trajectories inspired by ARCTraj (Kim et al. 2026), a dataset of real human task-solving trajectories collected via O2ARC 3.0 (Shim et al. 2024). From this data collection process, we observed that humans typically follow a correct trajectory up to an intermediate step and then suddenly deviate with an incorrect action. SOLAR-Generator models this error pattern by branching from expert trajectories at random intermediate steps and executing plausible but incorrect actions. Importantly, the injected actions are not uniformly random. Instead, they are constrained to semantically plausible alternatives, such as changing only the spatial extent of a selection or choosing a different color from the set of valid colors. This design produces suboptimal trajectories that reflect realistic human mistakes rather than arbitrary noise.

Flexible multi-task training and evaluation.

SOLAR provides approximately 20 task variants derived from ARC-AGI, and SOLAR-Generator can synthesize trajectories for these tasks on demand.

Explicit action-level state transitions for stage-wise analysis.

SOLAR represents task-solving trajectories at the action level, and we integrate ARCLE (Lee et al. 2024) to obtain explicit state transitions. This setup allows us to attribute performance degradation to specific stages (proposal, evaluation, selection, application) of the decision pipeline.

B.2 Operations in SOLAR

Coloring	0 Color0	1 Color1	2 Color2	3 Color3	4 Color4
	5 Color5	6 Color6	7 Color7	8 Color8	9 Color9
FloodFill	10 FloodFill0	11 FloodFill1	12 FloodFill2	13 FloodFill3	14 FloodFill4
	15 FloodFill5	16 FloodFill6	17 FloodFill7	18 FloodFill8	19 FloodFill9
Object Oriented	20 MoveU	21 MoveD	22 MoveR	23 MoveL	
	24 Rotate90	25 Rotate270	26 FlipH	27 FlipV	
Clipboard	28 CopyI	29 CopyO	30 Paste		
Critical	31 CopyInput	32 ResetGrid	33 ResizeGrid	34 Submit	35 None

Fig. 6: All operations compatible with SOLAR, 0–34 operations follow ARCLE, and only in SOLAR, 35 (**None**) is for terminated episode. This indicates that the episode ends after **Submit**.

The operations from 0 to 34 are identical to those used in ARCLE (Lee et al. 2024), as illustrated in Figure 6. Since **Submit** is an operation that receives a reward, it should only be used when the state is considered correct and not excessively. Due to LDCQ’s fixed horizon, and to ensure that the agent only uses **Submit** when the state is definitively correct, we added a **None** operation that fills all subsequent states after **Submit** with the 11th color (10), which does not exist in the original ARC (0–9). In other words, during training, the **None** action emphasizes that the episode ends after **Submit**.

B.3 Grid Maker

To generate SOLAR, we develop a SOLAR-Generator that synthesizes a large volume of data for a given rule. Grid Maker is a hard-coded program specific to each task. Grid Maker contains the rules for synthesizing demonstration examples and test examples, and the synthesized solution action path consists of operations and selections. In Grid Maker, data is formatted to be compatible with ARCLE. The Grid Maker constructs analogies with the same problem semantics but with various attributes, such as the shape, color, size, and position of objects. SOLAR-Generator can generate intermediate trajectories by interacting with ARCLE. The SOLAR-Generator algorithm is designed to augment specific tasks using the Grid Maker, which can be primarily divided into three parts.

Grid Maker was built as a data loader used in ARCLE. In the original ARCLE environment, there was no need to load operations and selections. Only the grid was loaded with the original ARC. To change this structure, the entire environment would need to be recreated. Instead, operations and selections are now loaded from the data loader’s description, thereby preserving the original environment. Therefore, the process of creating input-output examples and generating action sequences works within a single file.

Specifying Common Parts

Each task in the ARC dataset usually contains 3 demonstration examples, with common elements observed across these pairs. In the common parts, attributes such as color, task type, and object presence are determined by random values before pair generation.

Synthesizing Examples

In the example synthesis phase, the input of the original task is augmented in a way that ensures diversity while preserving the integrity of the problem-solving method. A random input grid is generated under conditions that satisfy the task’s required analogy. A solution grid is created using a hard-coded algorithm. For tasks involving pattern-based problems, as experimented in the paper, selections are made to fit the grid size, and various operations are executed either randomly or in a predetermined order. For object-based problems, the solution grid is generated by an algorithm that identifies the required objects in the input grid and processes them in accordance with the task requirements.

Converting to ARCLE Trajectories

This stage involves the creation of an ARCLE-based trajectory that closely follows the problem-solving schema of the synthesized examples. The entire process is implemented using a hard-coded algorithm. During the example synthesis process, object locations may already be known, or they can be identified using a search algorithm. The information obtained is used to make appropriate selections, and the trajectory is converted into an ARCLE trajectory via an algorithm that yields the correct solution.

Algorithm 1: SOLAR-Generator

Input: Task set T , grid size (H, W) , samples N , examples E , horizon H_{seg}
Output: Training and test trajectory datasets $\{\tau^{train}, \tau^{test}\}$

```
1 for  $task \in T$  do
2    $\mathcal{H}_{test} \leftarrow \emptyset$ ;
3   for  $type \in \{test, train\}$  do
4      $\mathcal{D}_s \leftarrow \text{GridMaker}(task, (H, W), N_{type}, E, seed_{type})$ ;
5     for  $(ex_{in}, ex_{out}, pr_{in}, pr_{out}, desc) \in \mathcal{D}_s$  do
6        $hash \leftarrow \text{SHA256}(pr_{in} || pr_{out})$ ;
7       if duplicate or ( $type = train$  and  $hash \in \mathcal{H}_{test}$ ) then
8         continue;
9       end
10       $\tau \leftarrow \emptyset, t \leftarrow 0$ ;
11       $s_0 \leftarrow \text{ARCLE.reset}(pr_{in})$ ;
12      for  $(opr_t, sel_t) \in (desc.ops, desc.sels)$  do
13         $a_t \leftarrow (opr_t, sel_t)$ ;
14         $s_{t+1}, r_t, done_t \leftarrow \text{ARCLE.step}(s_t, a_t)$ ;
15         $\tau \leftarrow \tau \cup \{s_t, a_t, r_t, done_t\}$ ;
16         $t \leftarrow t + 1$ 
17      end
18      if  $is\_expert$  and  $s_t.grid \neq pr_{out}$  then
19        continue;
20      end
21      Save  $\tau$  and  $\{\tau_i^{seg}\}$  with horizon  $H_{seg}$ ;
22    end
23    if  $type = test$  then
24       $\mathcal{H}_{test} \leftarrow$  current hashes;
25    end
26  end
27 end
28 return  $\{\tau^{train}, \tau^{test}\}$ ;
```

Declarations

Funding: This work was supported by NRF (RS-2024-00451162; 50%), IITP (RS-2023-00216011; 20%, 2019-0-01842; 10%) and GIST (KH0870, Future-leading Specialized Research Project; 20%) grants funded by the Ministry of Science and ICT, Korea. Computing resource were supported by GIST SCENT-AI, AICA, and KISTI.

Competing interests: The authors have no competing interests to declare.

Code Availability: Codes are available at https://github.com/GIST-DSLAb/solar_ldcq.

References

- Chollet F (2019) On the Measure of Intelligence. arXiv:191101547
- Fu J, Kumar A, Nachum O, et al (2020) D4RL: Datasets for Deep Data-Driven Reinforcement Learning. arXiv:200407219
- Fujimoto S, van Hoof H, Meger D (2018) Addressing Function Approximation Error in Actor-Critic Methods. In: ICML
- Hang T, Gu S, Li C, et al (2023) Efficient Diffusion Training via Min-SNR Weighting Strategy. In: ICCV
- Ho J, Jain A, Abbeel P (2020) Denoising Diffusion Probabilistic Models. In: NeurIPS
- Hodel M (2024) Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:240407353
- Kim S, Choi H, Lee S, et al (2026) ARC'Traj: A Dataset and Benchmark of Human Reasoning Trajectories for Abstract Problem Solving. In: KDD
- Kostrikov I, Nair A, Levine S (2022) Offline Reinforcement Learning with Implicit Q-Learning. In: ICLR
- Kumar A, Zhou A, Tucker G, et al (2020) Conservative Q-Learning for Offline Reinforcement Learning. In: NeurIPS
- Laird JE, Newell A, Rosenbloom PS (1987) Soar: An Architecture for General Intelligence. *Artificial Intelligence* 33(1):1–64
- Lee H, Kim S, Lee S, et al (2024) ARCLE: The Abstraction and Reasoning Corpus Learning Environment for Reinforcement Learning. In: CoLLAs
- Li WD, Hu K, Larsen C, et al (2024) Combining induction and transduction for abstract reasoning. arXiv:241102272
- Schaul T, Quan J, Antonoglou I, et al (2016) Prioritized Experience Replay. In: ICLR
- Shim S, Ko D, Lee H, et al (2024) O2ARC 3.0: A Platform for Solving and Creating ARC Tasks. In: IJCAI, URL <https://o2arc.com>
- Song J, Meng C, Ermon S (2020) Denoising Diffusion Implicit Models. In: NeurIPS
- Venkatraman S, Khaitan S, Akella RT, et al (2024) Reasoning with Latent Diffusion in Offline Reinforcement Learning. In: ICLR